Основи на модерния дизайн на клас.

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

Разделна компилация. default/delete на специални функции (C++ 11). Статични член данни. Енумерации. Namespaces.

Разделна компилация

Какво е разделна компилация?

.h → декларации
.cpp → имплементация

Предимства:
по-бърза компилация
по-чист дизайн
независими модули

Student.h

```
#pragma once
#include <string>

class Student {
    std::string name;
public:
    Student(const std::string& n);
    std::string getName() const;
};
```

//В C++ 20 се въвеждат модулите
// за да решат проблеми - като multiple includes in headers

Student.cpp

```
#include "Student.h"
Student::Student(const std::string& n) :
name(n) {}

std::string Student::getName() const {
    return name;
}
```

Грешки при използване на header-и

```
// Student.h
class Student {
public:
    void print() const;
};

// Student.cpp
   void Student::print() {   // ❌ липсва
const
}
```

Добри практики

✔ .h = интерфейс
✔ .cpp = имплементация
✔ Минимални include-и
✔ Forward declarations
✔ Никакви .cpp includes
✔ Никакъв using namespace в header

```
-> Circular dependencies -> forward declaration
// A.h
#include "B.h"
class B;
class A {
    B b;
};

// B.h
#include "A.h"

class B {
    A a;
};
```

Какво е препроцесор?

Работи преди компилация
Основни директиви:
#include
#define
#ifdef / #ifndef

Условна компилация:

```
#ifdef DEBUG
std::cout << "Debug mode\n";
#endif
// полезна за дебъгване
```

Добри практики

```
#ifndef STUDENT_H
#define STUDENT_H
// код
#endif
};
```

```
#define PI 3.14        //лоша практика
constexpr double PI = 3.14; // добра практика
```

Обяснение на разделната компилация

Една C++ програма може да бъде разделена на множество изходни файлове (.cpp), които се компилират независимо един от друг – това се нарича разделна компилация.

Преди същинската компилация всеки изходен файл се обработва от препроцесора, който изпълнява всички директиви, започващи със символа #.

Например, при всяка среща на директивата #include препроцесорът я заменя със съдържанието на съответния хедър файл, който обикновено съдържа декларации.

В резултат на компилацията на всеки .cpp файл се получава отделен обектен файл (с разширение .obj), съдържащ машинен код.

Изпълнимият файл на програмата (с разширение .exe) се създава на следващ етап – свързване (linking) – при което линкерът обединява всички обектни файлове.

Линкерът свързва всички референции към имена (променливи, функции, класове и др.) от даден обектен файл със съответните им дефиниции, които могат да се намират в други обектни файлове.

Възможно е дадена дефиниция да липсва във всички обектни файлове. В такъв случай линкерът я търси в стандартната C++ библиотека, стандартната C библиотека, както и във всички допълнително указани от програмиста библиотеки. Ако дефиницията не бъде открита никъде, линкерът генерира грешка.

Какво е CMake?

CMake е инструмент, който генерира build файлове
(Makefile, Visual Studio, Ninja и др.) от билд конфигурация.

❓ Защо не компилираме директно с g++?

1)проектът има много .cpp файлове

2)различни платформи (Linux / Windows / macOS)

3)различни компилатори

4) “Правилният” начин да билднете проект

Основна идея

➡️ Описваме проекта веднъж (CMakeLists.txt)
➡️ CMake генерира подходящ build
➡️ компилаторът върши останалото

```
project/
  ├── CMakeLists.txt
  ├── main.cpp
  ├── Student.h
  └── Student.cpp
```

Пример

Кога да ползваме =delete?

=delete е добра идея винаги, когато трябва изрично да забраним определена операция, защото тя няма смисъл, е опасна или нарушава дизайна на класа.

Пример:

Класът не трябва да се копира (owning ресурс) - най-честият и правилен случай.

Класът притежава уникален ресурс: файл, сокет, mutex, GPU ресурс, OS handle
Копиране би довело до double free / undefined behavior.

```
class File {
    FILE* f_;
public:
    File(const char* name) : f_(fopen(name, "r"))
{}  ~File() { if (f_) fclose(f_); }

    File(const File&) = delete;
    File& operator=(const File&) = delete;

    File(File&&) noexcept = default;
    File& operator=(File&&) noexcept = default;
};
```

Още примери за =delete

Примери

```
class Student {
public:
    Student(int id);
    Student(double) = delete;   //
забраняваме неясни конверсии
};
```

Примери

```
class Student {
    std::string name_;
public:
    Student() = delete;
    explicit
Student(std::string name) :
name_(std::move(name))
{}
};
```

когато искаме класът няма смисъл без дефолтни данни.

Примери за функции

```
void log(int) = delete;
void log(double) = delete;

void log(const std::string& msg) {
    std::cout << msg;
}
```

Какво са статични член данни?

Член-променлива, споделена между всички инстанции на класа.

Декларира се в класа; дефинира се (и, при нужда, инициализира) извън него — с изключение на inline static (C++17+).

Статичните член-данни имат живот, различен от обектите: инициализират се преди main() или при първия използван translation unit (implementation-defined ordering).

```cpp
class MyClass {
  public:
  static int s_count; // декларация
  MyClass();
};


// source MyClass.cpp
int MyClass::s_count = 0; // дефиниция + инициализация


struct Widget {
  inline static int s_count = 0;
  // C++17+: дефиниция в header
  static constexpr int MAX = 256;
  //константа известна на компилатора
};

//constexpr - смята се по време на компилация
```

Проблеми при статичните данни

Редът на инициализация на статични обекти в различни translation units е неопределен.

Това води до бъгове, когато един статичен обект използва друг статичен обект от друг .cpp файл.

```
extern int valueB;
int valueA = valueB + 1; // може да е неинициализирана

// B.cpp
int valueB = 42;
```

Решение

```
int& safeValue() {
    static int value = 42; // lazy + thread-safe
    return value;
}
//Scott Meyers - пример за Singleton
```

Практически съвети

Избягвайте глобални и статични обекти със сложна логика.
Предпочитайте constexpr, inline static или function-local static.
Ако редът има значение – контролирайте го чрез функции.

Енумерации

Енумерацията е отделен тип, чиято стойност е ограничена до диапазон от стойности, който може да включва няколко изрично посочени константи(енумератори).

Стойностите на константите са стойности от интегрален тип, известен като основен тип на енумерацията.

Енумерацията има същия размер, представяне на стойност и като неговия основен тип. Освен това всяка стойност на енумерацията има същото представяне като съответната стойност на основния тип.

Пример

```
enum IceCream1 {
    vanilla, //0
    chocolate, //1
    strawberry, //2
    mango, //3
    oreo //4
};

enum IceCream2 : char {
    vanilla, //0
    chocolate, //1
    strawberry, //2
    mango, //3
    oreo //4
};

enum class Animal { dog, deer, cat, bird, human };
```

Пример за енумерации

```
enum Color { red, green, blue };                    // plain enum
enum Card { red_card, green_card, yellow_card };    // another plain enum
enum class Animal { dog, deer, cat, bird, human };  // enum class
enum class Mammal { kangaroo, deer, human };        // another enum class

void example() {
    // examples of bad use of plain enums:
    Color color = Color::red;
    Card card = Card::green_card;

    int num = color;    // no problem

    if (color == Card::red_card) // no problem (bad)
        cout << "bad" << endl;

    if (card == Color::green)   // no problem (bad)
        cout << "bad" << endl;

    // examples of good use of enum classes (safe)
    Animal a = Animal::deer;
    Mammal m = Mammal::deer;

    int num2 = a;   // error
    if (m == a)     // error (good)
        cout << "bad" << endl;

    if (a == Mammal::deer) // error (good)
        cout << "bad" << endl;
}
```

Namespaces

```
namespace A {
    int i;
}
namespace B {
    int i;
    int j;

    namespace C {
        namespace D {
            using namespace A; // all names from A injected into global namespace

            int j;
            int k;
            int a = i;         // i is B::i, because A::i is hidden by B::i
        }

        using namespace D; // names from D are injected into C
                           // names from A are injected into global namespace

        int k = 89; // OK to declare name identical to one introduced by a using
        int l = k;  // ambiguous: C::k or D::k
        int m = i;  // ok: B::i hides A::i
        int n = j;  // ok: D::j hides B::j
    }
}
/*Пространствата от имена предоставят метод за предотвратяване на конфликти с имена.
Символите, декларирани вътре в namespace block, се поставят в наименуван scope, който не
позволява да бъдат сбъркани със символи с идентични имена в други диапазони.*/
```

Пример

```cpp
#pragma once
#include <string>
#include <iostream>

namespace School {

enum class GradeLevel { Freshman, Sophomore, Junior, Senior };

class Student {
private:
    std::string name;
    int age;
    GradeLevel grade;

    static int studentCount; // брояч на студентите

public:
    // Конструктори
    Student() = default; // default конструктор
    Student(const std::string& n, int a, GradeLevel g);

    // Забраняваме конструктор с 1 параметър (пример за delete)
    Student(int a) = delete;

    ~Student();

    void print() const;

    static int getStudentCount() { return studentCount; }
};

} //
```

Пример

```cpp
#include "Student.h"

namespace School {

// Инициализация на статичния член
int Student::studentCount = 0;

// Конструктор с параметри
Student::Student(const std::string& n, int a, GradeLevel g)
    : name(n), age(a), grade(g)
{
    ++studentCount;
}

// Деструктор
Student::~Student() {
    --studentCount;
}

// Метод за печат
void Student::print() const {
    std::cout << "Name: " << name
              << ", Age: " << age
              << ", Grade: ";
    switch (grade) {
        case GradeLevel::Freshman: std::cout << "Freshman"; break;
        case GradeLevel::Sophomore: std::cout << "Sophomore"; break;
        case GradeLevel::Junior: std::cout << "Junior"; break;
        case GradeLevel::Senior: std::cout << "Senior"; break;
    }
    std::cout << '\n';
}

} // namespace School
```

Въпроси?

Благодаря за вниманието!